



الجامعة اللبنانية
UNIVERSITE LIBANAISE



IN410

Cryptography and Secure Communications

Ahmad Fadlallah

Credits



- The following slides are the adaptation of
 - Lecture slides of Lawrie Brown based on William Stallings book “Cryptography and Network Security: Principles and Practice”
 - Lecture Slides of Dan Boneh – Stanford University
 - Lecture slides of Pawel Wocjan – University of Central California
 - Lecture Slides of Ahmed Serhrouchni – Telecom ParisTech
 - Others (References in the note section)

Course Syllabus



- Context
- History/ Classical ciphers
- **Symmetric Encryption ←**
- Asymmetric Encryption
- Cryptographic hash functions
- Key Distribution
- Security Protocols



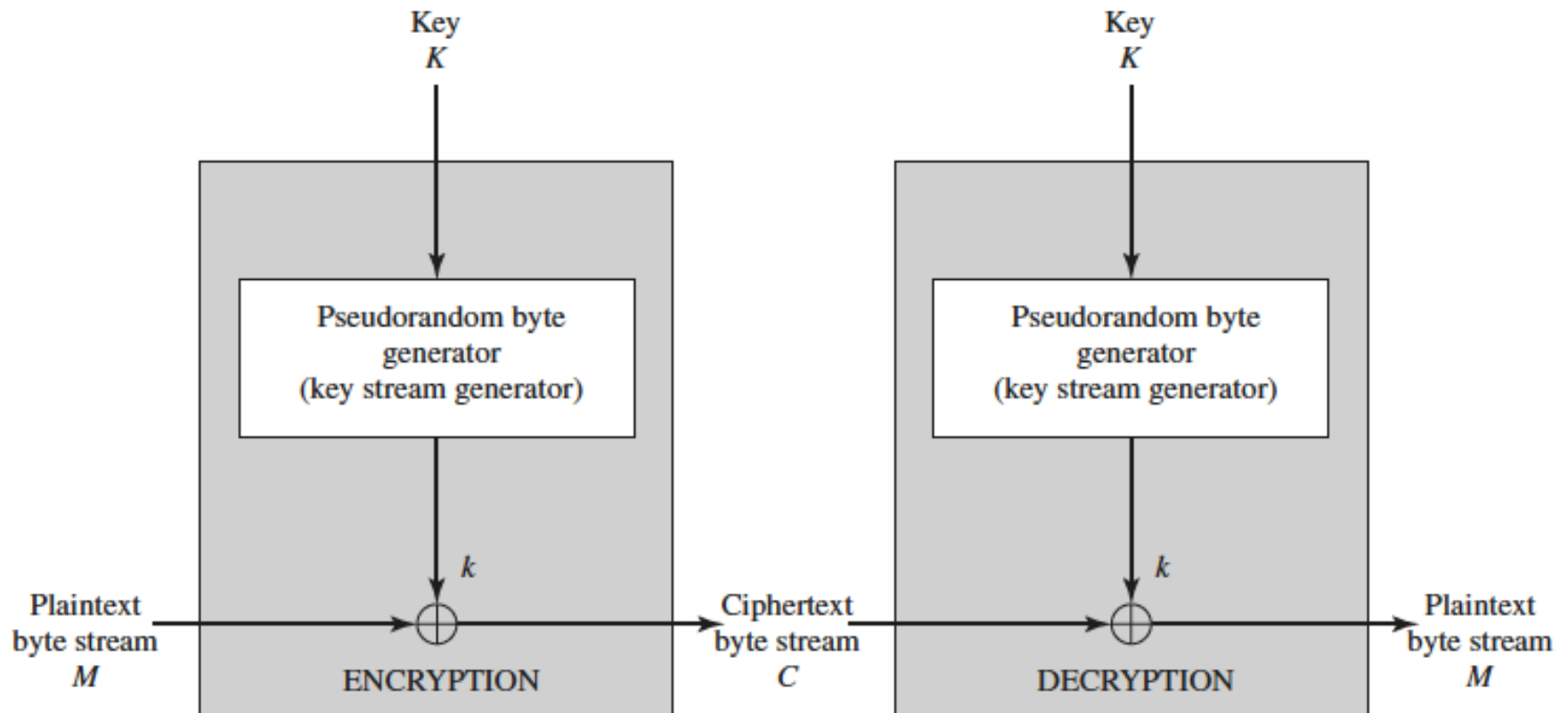
Symmetric Encryption

- Block Ciphers
- **Stream Ciphers** ←
- Operation Modes



Stream Ciphers

Introduction



Rivest Cipher 4 (RC4)



- Designed in 1987 by Ron Rivest for RSA Security.
- Variable key size stream cipher with byte-oriented operations.
- Based on the use of a random permutation



RC4: How it Works?

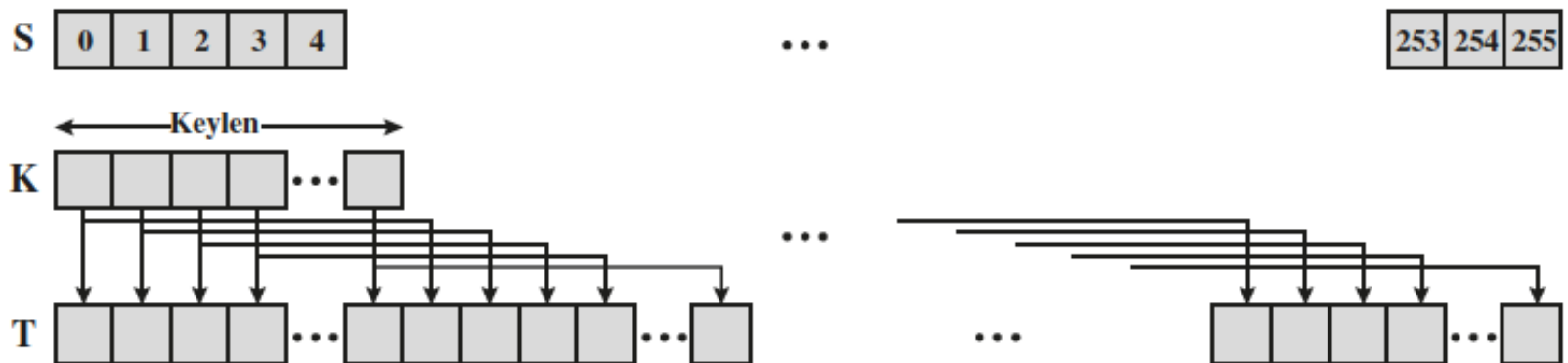
- A **variable-length key** of from 1 to 256 bytes (8 to 2048 bits) is used to initialize a **256-byte state vector S** , with elements $S[0], S[1], \dots, S[255]$.
- S always contains **a permutation of all 8-bit numbers** from 0 through 255.
- For encryption and decryption, **a byte k** is generated from S by selecting one of the 255 entries in a **systematic** fashion.
- **As each value of k is generated, the entries in S are once again permuted**



RC4: How it Works?

- S: State Vector, T: Temporary Vector
- Initialization:

```
for i = 0 to 255 do
    S[i] = i;
    T[i] = K[i mod keylen];
```

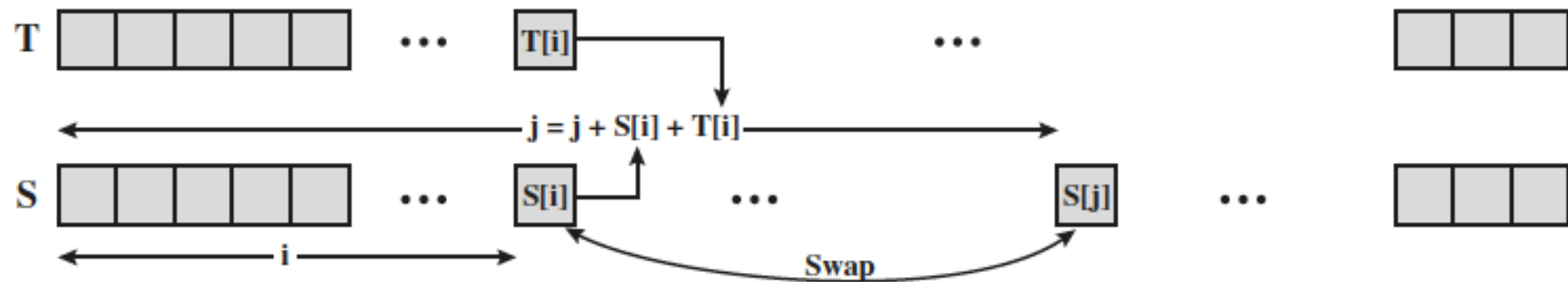




RC4: How it Works?

- Initial Permutation of S

```
j = 0;  
for i = 0 to 255 do  
    j = (j + S[i] + T[i]) mod 256;  
    Swap (S[i], S[j]);
```





RC4: How it Works?

- Stream Generation

```
i, j = 0;
```

```
while (true)
```

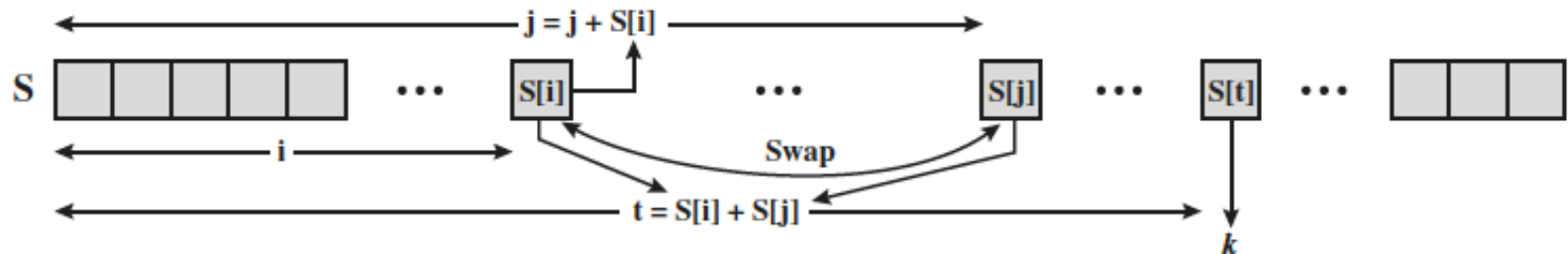
```
    i = (i + 1) mod 256;
```

```
    j = (j + S[i]) mod 256;
```

```
    Swap (S[i], S[j]);
```

```
    t = (S[i] + S[j]) mod 256;
```

```
    k = S[t];
```



RC4 (In)Security



- **Vulnerabilities** have been discovered over time.
- These include **biases** in the output stream that make it susceptible to attacks
- Many attacks have been demonstrated
- **Not considered anymore as secure Algorithm**

Other Stream Ciphers – Salsa20



- High-speed stream cipher with proven security.
- Uses a 256-bit key.
- Key Features:
 - Simplicity and high performance.
 - Provably secure under well-defined conditions.
- Largely replaced by ChaCha20 BUT is still considered secure and is used in some legacy systems.

Other Stream Ciphers – ChaCha20



- Designed by Daniel J. Bernstein
- *ChaCha20* is an improved variant of the *Salsa20* stream cipher.
- Known for its high performance, simplicity, and resistance to cryptanalysis.
- Works with 256-bit keys.
- Efficient on both hardware and software

Other Stream Ciphers – HC-256



- Designed for high-speed software applications.
- Typically used in high-performance computing environments.
- Key Features:
 - Uses a 256-bit key
 - High throughput and secure against known attacks.

Other Stream Ciphers – Grain



- A **lightweight** stream cipher designed for constrained environments like IoT devices.
- It **balances** *security* with *resource efficiency*.
- Common in embedded systems and wireless communication.
- Key Features:
 - Supports 80-bit (Grain v1) and 128-bit (Grain-128) keys.
 - **Energy-efficient** design for low-power devices.

Other Stream Ciphers – Trivium



- Designed for lightweight applications such as RFID and IoT devices
- Key Features:
 - Very small implementation footprint.
 - Secure for known cryptanalytic attacks.



Conclusion: Stream Ciphers

- Stream ciphers:
 - Encrypt data as a continuous flow
 - Use keystream $\oplus$ plaintext (XOR)
- Provide High speed and Low latency
- Modern Designs: Salsa20, ChaCha20
- Lightweight Algorithms (for constrained devices)
 - Trivium, Grain

Conclusion: Stream Ciphers



- Key Observations
 - Most designs stabilized around ~2008 (eSTREAM project)
 - Very few new standalone stream cipher standards
- The need for stream encryption remains, but the approach has evolved

From Stream Ciphers to Operation Modes



- Shift in Cryptographic Design: Instead of designing many ciphers, Use one strong primitive: AES
- New Requirements
 - Beyond encryption (confidentiality): Integrity, Authentication
- How can we use a block cipher to:
 - Encrypt long or streaming data?
 - Achieve stream-like behavior?
 - Provide stronger security guarantees?
- We do not design a new cipher... We design how to use an existing one → Modes of Operation



Simplified RC4 Stream Cipher - Exercise